# Efficient Time-Series Database Implementation for High-Frequency Traffic Sensor Data

Xin Yi Oon
College of Information and Communication Engineering
Ajou University
Suwon, Korea
anneirist@ajou.ac.kr

Chen Qian Xu
College of Information and Communication Engineering
Ajou University
Suwon, Korea
xuchenqian@ajou.ac.kr

Yu Xiang Tong
College of Information and Communication Engineering
Ajou University
Suwon, Korea
tongyuxiang0208@gmail.com

***Abstract*—This paper presents the implementation of a time-series database (TSDB) for managing high-volume traffic sensor data generated by SUMO simulations. We use TimescaleDB, a PostgreSQL extension, to handle time-stamped metrics such as flow, occupancy, and average speed. A Python-based data ingestion pipeline processes SUMO's detector_output.xml and inserts data into the TSDB at 1-second intervals. Data retention policies were configured to keep raw data for 7 days before aggregation and deletion, and compression was applied to improve storage efficiency. The system supports range queries, interpolation, gap filling, and rollup aggregations to compute 5-minute averages in a hypertable. Overall, this setup efficiently stores and queries continuous traffic data, demonstrating the advantages of TSDBs for simulation-based traffic analysis.. (*Abstract*)**

***Keywords—Time-series database; TimescaleDB; Post-greSQL; SUMO; Traffic simulation; Data ingestion; Query op- timization.****(key words)*

## I. Introduction

The increasing availability of traffic sensor data, combined with the need for real-time traffic monitoring and management, has led to significant challenges in data storage, retrieval, and analysis. Modern traffic simulations, such as those generated by the Simulation of Urban Mobility (SUMO) platform, produce high-frequency time-stamped datasets that capture vehicle flow, occupancy, speed, and other traffic parameters. Handling such continuous streams of data requires a database system that can efficiently store, query, and analyze temporal information without compromising performance.

While traditional relational databases are robust, they often struggle with high-frequency time-series data due to the overhead of indexing, partitioning, and query execution over large datasets. Time-series databases (TSDBs) have emerged to address these challenges, offering features such as automatic partitioning, compression, and time-oriented query optimization. Among these, TimescaleDB, a PostgreSQL extension, provides scalable, high-performance time-series data management while retaining the relational capabilities familiar to many developers.

In this work, we implement a TimescaleDB-based solution to efficiently manage traffic sensor data generated by SUMO simulations. A Python-based ingestion pipeline is developed to process SUMO's detector_output.xml files and insert time-stamped data into the database at 1-second intervals. The system supports a variety of analytical queries, including range queries, interpolation of missing data, gap detection, and aggregation over fixed intervals. Data retention and compression policies are also employed to optimize storage while preserving critical information.

The main contributions of this paper are as follows:

•A practical implementation of a high-frequency time- series database for SUMO-generated traffic data using TimescaleDB.

•A Python-based ingestion pipeline capable of efficiently parsing, transforming, and inserting large volumes of sensor data.

•Demonstration of advanced time-series query techniques including interpolation, gap detection, and aggregation, highlighting the performance and storage benefits of using a TSDB.

This paper is structured as follows. Section II presents the system design and details of TSDB selection. Section III describes evaluates the system ' s performance, storage efficiency, and query accuracy. Finally, Section IV concludes the work and outlines potential directions for future research.

## II. System design and TSBD selection

### A. System Overview

The system efficiently collects, stores, and analyzes high- frequency traffic sensor data from SUMO simulations. It consists of three main components: the traffic simulation module, the data ingestion pipeline, and the time-series database.

1)The traffic simulation is executed using SUMO via the command line (sumo -c simulation.sumocfg) or with the GUI (sumo-gui -c simulation.sumocfg). Three traffic detectors are deployed on lane edge1_0 at positions of 100 m, 500 m, and 900 m. Each detector records vehicle count, flow, occupancy, and speed at 1-second intervals. The collected data are subsequently aggregated and exported to detector_output.xml for further analysis.

2) Data Ingestion Pipeline: A Python pipeline parses the XML output, converts simulation times to timestamps, and inserts data into the database in batches for efficiency.

3) Time-Series Database: TimescaleDB, an extension of PostgreSQL, stores the data in hypertables with indexing, compression, and aggregation support, enabling fast queries and efficient storage of high-frequency traffic data.

### B. TSDB Setup and Configuration

The time-series database (TSDB) was implemented using PostgreSQL with the TimescaleDB extension to provide scalability and efficient time-based data management. The setup process was automated through a Python script that utilized the psycopg2 library for database connectivity and configuration.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

1) Database Initialization: The script first established a connection to the PostgreSQL server hosted locally on port 5433 using the credentials of the database administrator. If an existing database named traffic_data was detected, it was dropped and re-created to ensure a clean setup environment:

```
cursor .execute ("DROP DATABASE IF EXISTS traffic_data")
cursor .execute("CREATE DATABASE traffic_data")
```

2) Enabling TimescaleDB Extension: After establishing a new connection to the traffic_data database, the TimescaleDB extension was enabled to allow time-series functionality:

```
cursor .execute("CREATE EXTENSION IF NOT EXISTS timescaledb")
```

This extension enables PostgreSQL to efficiently manage time- stamped data using hypertables, chunking, and compression features.

3) Table Creation: A relational table named detector_data was created to store traffic sensor readings. The table includes fields such as time, detector_id, flow, occupancy, speed, and length, corresponding to data extracted from SUMO's detector_output.xml file:

```
CREATE TABLE IF NOT EXISTS detector_data
( time TIMESTAMPTZ NOT NULL,detector_id VARCHAR(50) NOT NULL, nVehContrib INTEGER,
flow DOUBLE PRECISION,occupancy DOUBLE PRECISION,speed DOUBLE PRECISION, length DOUBLE PRECISION,nVehEntered INTEGER );
```

4) Conversion to Hypertable: To optimize for time-series workloads, the standard relational table was converted into a TimescaleDB hypertable, with time as the primary time dimension and a chunking interval of one day:

```
SELECT create_hypertable( 'detector_data', 'time', if_not_exists => TRUE, chunk_time_interval => INTERVAL '1 day' );
```

This conversion allows PostgreSQL to automatically partition the data by time, improving query performance and scalability for high-volume sensor data.

5) Indexing: An index was created on the detector_id and time columns to accelerate query performance, especially for time-range and sensor-specific lookups:

```
CREATE INDEX IF NOT EXISTS idx_detector_time ON detector_data (detector_id, time DESC);
```

After completing these steps, the database was fully config- ured as a time-series optimized system. The script concluded with instructions for the next stages: running the SUMO simulation, ingesting data via Python, and performing analytical queries.

### C. *Data Ingestion*

The data ingestion process is responsible for importing traffic detector data from a SUMO simulation output file into a PostgreSQL database. This process ensures that raw XML data is transformed into a structured format suitable for further analysis.

1) Database Connection: A connection to the PostgreSQL database is established using psycopg2, and a cursor is created for executing SQL commands.

2) Parsing XML Data: The SUMO output file detector_output .xml is parsed using the xml .etree .ElementTree library. Each <interval> element contains traffic data for a specific time interval and detector.

```
tree = ET .parse('detector_output .xml')
root = tree.getroot()
intervals = root .findall('interval')
```

3) Extracting and Transforming Records: For each interval, the following attributes are extracted:

- begin_time– start time in seconds
- detector_id– detector ID
- nVehContrib – number of contributing vehicles
- flow – traffic flow rate
- occupancy – detector occupancy ratio
- speed – average vehicle speed
- length – average vehicle length
- nVehEntered – number of vehicles entered

The simulation time is converted to an actual timestamp by adding the begin offset to the current system time:

```
actual_time = start_time + timedelta(seconds=begin_time)
```

4) Batch Insertion into Database: Records are temporarily stored in memory and inserted in batches of 1000 for efficiency. The SQL insertion uses placeholders for the extracted values:

```
INSERT INTO detector_data
(time, detector_id, nVehContrib, flow, occupancy, speed,length, nVehEntered)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
```

5) Summary and Cleanup: After all records are inserted, asummary is printed showing the total number of records and the count per detector. Finally, the database connection is closed properly.

This ingestion process ensures efficient and accurate trans- fer of SUMO simulation data into a structured database for downstream analysis and visualization.

*D. Query Implementation*

The query implementation involves retrieving, processing, and analyzing traffic detector data stored in the PostgreSQL database. This subsection describes four types of queries: range queries, interpolation, data gap detection, and aggregation. A connection to the PostgreSQL database is established using the psycopg2 library, and a cursor object is created to execute SQL statements efficiently.

Then, to satisfy the project requirement for data retention and compression, we set up a rule in TimescaleDB that keeps raw sensor data for only seven days.

```
SELECT add_compression_policy('detector_data',
        INTERVAL '7 days');
```

After the data becomes older than seven days, the database automatically compresses it to save storage space.

```
SELECT add_compression_policy('detector_data',
        INTERVAL '7 days');
```

We enabled compression on the detector_data table by grouping data by detector_id and ordering it by time, which makes the compression more efficient.

```
ALTER TABLE detector_data SET (
        timescaledb.compress,
        timescaledb.compress_segmentby = 'detector_id',
        timescaledb.compress_orderby = 'time DESC'
      );
```

We also manually compressed all existing data to show the effect immediately. After applying these settings, the storage size was reduced clearly, showing that our retention policy and compression setup work as expected.

```
SELECT compress_chunk(i) FROM
    show_chunks('detector_data') i;
```

1) Query 1: Range Query for Maximum Flow: This query retrieves the maximum traffic flow recorded by a specific detector (e.g., detector_1) over the last 4 hours. The steps include:

- Fetching the latest timestamp from the database.
- Computing the timestamp 4 hours prior.
- Retrieving the record with the highest flow within this time range.
- Calculating basic statistics: average, maximum, and minimum flow.

```
SELECT
    COUNT(*) as records,
    AVG(flow) as avg_flow,
    MAX(flow) as max_flow,
    MIN(flow) as min_flow
  FROM detector_data
  WHERE detector_id = 'detector_1'
    AND time >= %s
    AND time <= %s
```

2) Query 2:Linear interpolation is used to estimate traffic speed at a specific target time for a given detector (e.g., detector_2):

- For linear interpolation at a target timestamp, the two nearest data points surrounding the target were first identified, capturing both time and speed values.
- The time difference between these points, denoted as `time_span_seconds`, and the elapsed time from the earlier point to the target, `time_to_before`, were calculated.
- The interpolation ratio was then computed as :

  ratio = time_to_before / time_span_seconds

Using this ratio, the speed at the target timestamp was estimated according to the linear interpolation formula:

$$v_{\text{interpolated}} = v_{\text{before}} + (v_{\text{after}} - v_{\text{before}}) \times \text{ratio}$$

- The interpolated speed and the corresponding ratio were reported. If the two surrounding points occurred at the same timestamp, interpolation was unnecessary.

3) Query 3 Data Gap Detection and Filling evaluates the data integrity of traffic detector detector_3 and provides suggested values for filling missing data. The process can be summarized as:

- Detection of Data Gaps: Data gaps were identified using SQL's LAG function, which allows each record to be compared with its preceding entry to retrieve the previous timestamp and speed. The time difference between consecutive records (gap_seconds) was calculated, and intervals exceeding 10 seconds were flagged as gaps. These gaps were then ranked by duration, with the five largest gaps selected for reporting, providing a concise overview of missing data within the dataset.

```
WITH time_gaps AS ( SELECT
        time as current_time,
        LAG(time) OVER (ORDER BY time) as
            prev_time,
        speed as current_speed,
        LAG(speed) OVER (ORDER BY time) as
    prev_speed, EXTRACT(EPOCH FROM (time -
    LAG(time) OVER (ORDER BY time))) as
    gap_seconds FROM detector_data WHERE
    detector_id = 'detector_3' )
        SELECT prev_time, current_time, prev_speed,
        current_speed, gap_seconds FROM time_gaps
        WHERE gap_seconds > 10
ORDER BY gap_seconds DESC LIMIT 5
```

- Reporting Gaps and Filling Suggestions: For each detected gap, the system shows the time range, how long the gap lasts, and the speeds just before and after it. Missing data can be estimated by averaging these two

speeds. If no gaps are longer than 10 seconds, the dataset is considered complete and reliable.

4）Query 4: Aggregation and Temporal Analysis: Traffic data are aggregated over fixed time intervals (e.g., 5 minutes) to compute metrics such as average flow, maximum flow, and total vehicle count. This aggregation reduces data volume and enables efficient analysis of temporal traffic patterns.

```
SELECT time_bucket('5 minutes', time)
as bucket,detector_id, AVG(flow) as avg_flow,
MAX(flow) as max_flow, SUM(nVehEntered) as
total_vehicles FROM detector_data GROUP BY bucket,
detector_id ORDER BY bucket DESC LIMIT 10
```

5) Summary: The query implementation provides a comprehensive toolkit for:

- Retrieving key statistics over specific time ranges.
- Estimating missing data through interpolation.
- Detecting and suggesting corrections for data gaps.
- Aggregating data for efficient analysis and visualization.

## III. Results and Evaluation

### A. Performance and Compression

The data ingestion process successfully imported a total of 10,800 records from the SUMO detector output file detector_output.xml. The distribution of records among the detectors is shown in Table I.

| Detector | Number of Records |
|---|---|
| detector_1 | 3600 |
| detector_2 | 3600 |
| detector_3 | 3600 |

TABLE I

NUMBER OF RECORDS INSERTED PER DETECTOR.

To improve storage efficiency and query performance, a 5- minute aggregation was performed on the data. The compression results are summarized in Table II.

| Metric | Original | Aggregated | Compression (%) |
|---|---|---|---|
| All detectors | 10,800 | 39 | 99.6 |

TABLE II

DATA COMPRESSION AFTER 5-MIN AGGREGATION.

The aggregation significantly reduces the number of records while preserving the essential statistical information, enabling efficient storage and faster query execution.

### B. Query Efficiency

To evaluate the system's efficiency and accuracy, several representative queries were executed:

• Maximum flow (Detector 1, last 4 hours): The peak flow of 3,600 vehicles/hour occurred at 13:07:07 UTC. The average flow over this period was 746 vehicles/hour, with no missing data, indicating high data integrity.

• Interpolated speed query (Detector 2): The query at a specific timestamp did not require interpolation, as the exact data point was available.

• 5-minute aggregation query: Aggregating the last 10 time buckets produced average flows between 604.6 and 840 vehicles, with maximum flows up to 3,600 vehicles/hour. Aggregation reduced the dataset from 10,800 records to 39, demonstrating improved query performance and storage efficiency.

Overall, these results confirm that the TimescaleDB implementation efficiently manages high-frequency traffic data, delivers accurate query results, and achieves significant compression for aggregated datasets.

## IV. Conclusion

This project presents the successful implementation of a high-performance time-series database using TimescaleDB on PostgreSQL to manage continuous traffic sensor data generated from SUMO simulations. A robust Python-based ingestion pipeline was developed, efficiently storing and handling 10,800 high-frequency records. The system demonstrated strong performance and utility through several key aspects:

• Data Lifecycle Management and Storage Efficiency: Data integrity was maintained with no significant gaps observed. Retention and compression policies, including continuous 5-minute rollup aggregations, achieved 99.6% storage reduction while preserving essential statistical information.

• Advanced Query Performance: Complex time-series queries, including range queries, interpolation, gap filling, and aggregation, were executed accurately and efficiently. Metrics such as maximum flow, interpolated speed, and aggregated statistics validated the system's suitability for real-time traffic monitoring.

• These results demonstrate that relational database robustness can be effectively combined with the performance requirements of high-frequency, high-volume sensor data.

Overall, the evaluation confirms that TimescaleDB provides a scalable and efficient platform for both storage and rapid analysis of traffic data, supporting a fully functional system for traffic monitoring and analysis.

Future work may explore automated real-time ingestion pipelines and the integration of advanced traffic prediction models to further enhance traffic management and decision-making capabilities.

## References


[1]Timescale, Inc. TimescaleDB Documentation. https://docs.timescale.com/

[2] P. Wagner et al., "Traffic and Mobility Simulation with SUMO," in Transportation Research Procedia, vol. 15, pp. 379-388, 2016.

[3] P. J. G. Box et al., "PostgreSQL: The World's Most Advanced Open Source Relational Database," PostgreSQL Global Development Group.